



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Documentación: Práctica 1

Rest API

https://github.com/luciatamarit/DAS_practicas.git

María Román Cantillana

Lucía Tamarit Barberan

Introducción:

El objetivo de la práctica ha sido el diseño e implementación de una REST API que constituye la base del proyecto final de la asignatura. Tal y como se indica en la guía, el proyecto consiste en el desarrollo de una aplicación tipo ChatGPT, en la que se construye un modelo de negocio en torno a un modelo de lenguaje de gran tamaño.

La API desarrollada permite que un usuario pueda registrarse en la plataforma, autenticarse mediante JWT, iniciar nuevos chats, recuperar conversaciones anteriores y eliminarlas. Además, la aplicación incorpora una lógica de negocio específica que limita el número de mensajes que cada usuario puede enviar al mes.

Para la implementación del backend se ha utilizado Django Rest Framework, aprovechando su sistema de serialización, gestión de vistas basadas en clases y autenticación mediante JSON Web Tokens. La integración con el modelo de lenguaje se ha realizado a través de Ollama, que se ejecuta como servicio dentro del entorno Docker y permite realizar llamadas al LLM desde el backend mediante la función proporcionada en el repositorio base.

En conjunto, esta práctica ha permitido implementar la lógica completa de negocio de una aplicación conversacional, dejando preparada la API para su consumo posterior por parte del frontend en las siguientes prácticas.

Organización del trabajo:

De acuerdo con lo establecido en la guía de la práctica, hemos creado en GitHub un milestone denominado "Sprint 1", con el objetivo de organizar y estructurar el desarrollo de la REST API durante esta primera fase del proyecto

En dicho milestone se incluyeron los diferentes issues correspondientes a las tareas técnicas. Cada issue fue asignado a la persona responsable y posteriormente cerrado una vez finalizada su implementación, dejando constancia formal del trabajo realizado y de la finalización del sprint.

El reparto de tareas se realizó atendiendo a bloques funcionales del sistema, buscando una distribución equilibrada y coherente con la arquitectura del proyecto.

Trabajo conjunto

- Diseño inicial del modelo de datos y revisión de la arquitectura general.
- Resolución de conflictos en la rama feature/p1.
- Pruebas finales completas del sistema desde cero utilizando docker compose up.
- Verificación del cumplimiento de los requisitos establecidos en el enunciado.
- Documentación de la práctica

María

- **Modelo Usage y reglas de negocio:**
 - Implementación del modelo Usage con relación 1-1 con User.
 - Desarrollo de la lógica de límite mensual de mensajes.
 - Implementación del mecanismo de reinicio automático cuando se supera la fecha de renovación.
 - Pruebas forzando valores en base de datos para validar el comportamiento del sistema.
- **Gestión de usuarios y validaciones:**
 - Implementación del endpoint de registro con doble contraseña.
 - Validación de contraseña (>8 caracteres, al menos una mayúscula y una minúscula)
 - Validación de username único y formato correcto de email.
 - Implementación de endpoints de perfil y cambio de contraseña.
- **Verificación de la web:**
 - Verificación de los endpoints principales a través de Swagger y llamadas por terminal. (incluyendo los campos a rellenar)

Lucía

- **Modelo Chats y ChatMessage:**
 - Implementación de los modelos Chat y ChatMessage.
 - Configuración de las relaciones 1-N descritas en el enunciado
 - Implementación del CRUD completo de chats y mensajes.
- **Integración LLM, incluyendo:**
 - Integración de la función chat_llm proporcionada en el repositorio base.
 - Construcción del contexto de conversación para cada chat.
 - Llamada al modelo a través de Ollama.
 - Almacenamiento de la respuesta generada por el modelo dentro de la base de datos.

Modelado ORM

En esta práctica se ha utilizado el ORM (Object-Relational Mapping) de Django, que es la capa que permite definir la base de datos mediante clases Python (models). Cada clase representa una tabla, cada atributo representa una columna, y las relaciones entre clases (ForeignKey, OneToOneField, etc.) se traducen automáticamente a relaciones entre tablas. Esto permite trabajar con la base de datos sin escribir SQL manualmente, manteniendo el modelo de datos coherente con el código y generando las migraciones de forma controlada.

El modelo de datos implementado sigue la estructura indicada, con cuatro entidades principales: User, Usage, Chat y ChatMessage. El objetivo es soportar un sistema conversacional multiusuario, donde cada usuario puede crear múltiples conversaciones y, dentro de cada conversación, almacenar el historial de mensajes.

En primer lugar, se utiliza el modelo User (autenticación de Django) como entidad central del sistema. Sobre este usuario se apoya tanto el control de acceso a los recursos (cada usuario solo puede ver y modificar sus propios chats) como la lógica de negocio del límite mensual de mensajes. La autenticación se realiza mediante JWT, pero el almacenamiento y gestión del usuario se mantiene dentro del esquema estándar de Django.

Para implementar la restricción de uso mensual, se definió el modelo Usage, que mantiene el estado de consumo de mensajes por usuario. Este modelo se relaciona con User mediante una relación 1–1, garantizando que cada usuario tenga como máximo un registro de uso asociado. Incluye los campos `messages_used`, `messages_limit` (y `reset_date`). Aunque en el README del backend se contemplaba una posible app separada `usage/`, decidimos integrar el modelo Usage dentro de la app `users` porque se trata de información directamente asociada al usuario y no de un módulo independiente. Esta decisión simplifica la estructura del proyecto sin afectar al cumplimiento de los requisitos, ya que el modelo mantiene su propia tabla y migración en la base de datos.

La entidad Chat modela cada conversación iniciada por un usuario. Contiene `title` y `created_at`, y se relaciona con el usuario mediante una relación 1–N, implementada como `ForeignKey(User)`. Esto permite que un usuario tenga múltiples chats, y además posibilita aplicar control de permisos fácilmente filtrando por `chat.user == request.user` en los endpoints correspondientes.

Por último, el historial conversacional se almacena en ChatMessage, que representa cada mensaje individual dentro de un chat. Se relaciona con Chat mediante otra relación 1–N (`ForeignKey(Chat)`), de modo que cada chat tiene múltiples mensajes ordenados por fecha. El modelo incluye el campo `role`, restringido a los valores "user" y "system", para distinguir entre mensajes enviados por el usuario y respuestas generadas por el sistema/LLM según el esquema exigido por la práctica. Además, incorpora `content` (texto del mensaje) y `created_at` para persistir el orden temporal de la conversación y permitir reconstruir el contexto que se envía al modelo de lenguaje en cada interacción.

Con este modelado ORM se consigue una base de datos consistente con la lógica de la aplicación: control de propiedad de recursos por usuario, persistencia de conversaciones y mensajes, y aplicación de reglas de negocio de uso mensual a través del modelo Usage.

Dificultades encontradas

Durante el desarrollo de la práctica surgieron diversas dificultades técnicas y organizativas, que abordamos hasta lograr un funcionamiento estable.

En primer lugar, una de las principales dificultades estuvo relacionada con la correcta implementación de la lógica de negocio relacionada con límite mensual de mensajes. Aunque el planteamiento inicial parecía sencillo, la comprobación simultánea de las condiciones “límite superado” y “fecha de renovación vencida” exigía un orden preciso en la evaluación de las reglas. Fue necesario realizar pruebas controladas modificando manualmente los valores de `messages_used` y `reset_date` en base de datos para verificar que el sistema bloqueaba correctamente cuando se alcanzaba el límite y, a la vez, permitía el envío del mensaje cuando la fecha de renovación ya había pasado, reiniciando tanto el contador como la fecha futura de renovación.

En segundo lugar, se presentaron dificultades en la documentación automática de los endpoints mediante Swagger. Inicialmente, algunas vistas basadas en `APIView` no mostraban correctamente el request body ni el response body. Esto nos obligó a revisar el uso de vistas genéricas (`CreateAPIView`, `UpdateAPIView`) y a ajustar los serializers para que la documentación generada reflejara fielmente la estructura de los datos enviados y recibidos.

Otra dificultad importante estuvo relacionada con la gestión del trabajo en paralelo mediante Git. Al trabajar ambas integrantes sobre la rama `feature/p1`, se produjeron conflictos en determinados archivos, especialmente en vistas y serializers compartidos. Fue necesario utilizar `git rebase`, resolver conflictos manualmente y establecer una coordinación más estricta para evitar modificaciones simultáneas en los mismos módulos. Este proceso permitió mejorar la organización del trabajo y reforzar el uso de buenas prácticas en control de versiones.

También se detectó un problema relacionado con el manejo de fechas con zona horaria en Django. En determinadas pruebas aparecieron advertencias sobre “naive datetime” en el campo `reset_date`. Para solucionarlo, fue necesario revisar el uso de `timezone.now()` en lugar de `datetime.now()` y asegurar que todas las fechas almacenadas fueran “timezone-aware”, garantizando coherencia con la configuración de Django y evitando comportamientos inesperados en el cálculo de renovaciones mensuales.

Por último, la integración con el modelo de lenguaje a través de Ollama fue un reto técnico adicional. Fue necesario construir correctamente el contexto de conversación enviando al modelo los últimos mensajes almacenados en la base de datos y, , guardar la respuesta generada respetando el esquema definido en el enunciado.

El campo role debía ajustarse exclusivamente a los valores “user” o “system”, lo que nos obligó a adaptar el flujo de guardado para que la respuesta del modelo se almacenara como “system” en lugar de utilizar otras etiquetas como “assistant”. De esta manera se garantizó la coherencia entre el diseño del modelo de datos y la implementación real del sistema conversacional.

En resumen, estas dificultades nos han hecho profundizar en el manejo de Django Rest Framework y en el trabajo colaborativo con control de versiones, fortaleciendo tanto las competencias técnicas como organizativas del equipo.

Desviaciones

Durante el desarrollo del Sprint 1 se produjeron algunas desviaciones respecto a la planificación inicial, aunque ninguna de ellas afectó al cumplimiento final de los requisitos establecidos en el enunciado

En primer lugar, la planificación formal del sprint mediante milestone e issues no la hicimos al inicio del desarrollo, sino que la documentamos una vez finalizada la implementación técnica. Aunque el trabajo sí estuvo distribuido de manera organizada entre nosotras, la formalización en GitHub (creación del milestone “Sprint 1” y cierre de issues asociados) la realizamos posteriormente para dejar constancia estructurada del proceso.

En segundo lugar, algunas decisiones técnicas iniciales fueron ajustadas durante el desarrollo. Por ejemplo, el endpoint de registro lo implementamos con un único campo de contraseña, pero posteriormente se adaptó para aceptar doble contraseña conforme a lo indicado en el enunciado. Esta modificación implicó la actualización del serializer sin romper la compatibilidad con el flujo ya existente.

Asimismo, la lógica del límite mensual de mensajes la refinamos tras las primeras pruebas. Inicialmente, el reinicio del contador no se persistía inmediatamente tras la actualización de la fecha de renovación, lo que obligó a reestructurar el orden de evaluación de las condiciones para garantizar que el comportamiento coincidiera exactamente con lo descrito en la práctica

Otra desviación estuvo relacionada con la integración del modelo de lenguaje. Aunque el repositorio base proporcionaba una función para la conexión con el LLM, tuvimos que ajustar el flujo de guardado de mensajes para mantener coherencia con el modelo de datos definido (donde el campo role debe ajustarse a los valores indicados en el esquema de la base de datos)

Finalmente, se dedicó más tiempo del previsto a la verificación exhaustiva del sistema desde cero mediante docker compose up, con el objetivo de asegurar que el entorno completo (backend y Ollama) se levantara correctamente en condiciones similares a las que utilizarán los profesores para la corrección

Aunque esto supuso un pequeño retraso respecto al cronograma inicial, permitió garantizar la estabilidad final del proyecto.

Otra desviación estuvo relacionada con la organización de las aplicaciones del proyecto. Inicialmente se contempló la creación de una app independiente usage/. Sin embargo, tal y como hemos explicado en el apartado de Modelado ORM, se optó por integrar el modelo Usage dentro de la app users, al tratarse de información ligada al usuario y no de un subsistema independiente. Esta decisión permitió simplificar la estructura del proyecto sin alterar la lógica funcional del sistema.

En conjunto, las desviaciones observadas no alteraron los objetivos del sprint, sino que contribuyeron a mejorar la robustez de la implementación y a asegurar el cumplimiento íntegro de los requisitos de la práctica.

Uso de IA

Durante el desarrollo de la práctica hemos hecho uso puntual de herramientas de inteligencia artificial como apoyo a la consulta y resolución de dudas técnicas. Estas herramientas se emplearon principalmente para aclarar conceptos específicos de Django Rest Framework, revisar posibles errores de configuración y contrastar soluciones ante incidencias concretas. No obstante, el diseño del modelo de datos, la implementación de la lógica de negocio y la estructura general del proyecto los hemos desarrollado, utilizando la IA como recurso complementario de consulta.